

Programming Assignment #2 (due in 2 weeks)

Inverted Files

Inverted files are the primary data structure to support the efficient determination of which documents contain specified terms. The objective of this assignment is to process the *RFA* corpus, much as in the first assignment. However, this time in addition to a dictionary you must also build an inverted file that contains a postings list for each dictionary term. Your implementation should model a useable real-world indexing system; in particular, this means that: (a) your inverted file structure must be written to disk as a binary file; (b) your dictionary must be written to disk¹; (c) for each word in the lexicon you must store an integer file offset to the corresponding on-disk posting list; and finally, (d) you should process the source text file only once (many real-world collections are so big that the cost of multiple scans is prohibitive).

Construction (60 points)

Process the collection and create the dictionary and inverted file. The docids are the ID fields in the <P> tags of the corpus. You may use more than one program for this assignment if you prefer. For example, you could write a program that reads the input file and writes out records like “apple 143 2” and “orange 4 3” to represent the fact that *apple* occurs in document 143 twice and *orange* occurs in document 4 three times. These records must be sorted (by term, then by docid) and you might use a separate program just for this. Finally, you have to write out the sorted entries as an inverted file. It may not be easiest to use three programs as in this example; however, in real-world systems, there is usually at least a scan that produces a temporary file and a merging of sorted temporary files. The reason for this is because typically records for the entire collection do not fit in the memory of a single machine. *However*, since our electronic texts are small it is quite possible to create an index using a single program without relying on auxiliary storage for this assignment. This can be achieved by following the memory-based inversion algorithm in the lectures (*Algorithm A*) and directly writing out the postings after all documents (*i.e.*, all paragraphs) have been read.

For full credit your inverted file should be a binary file². I suggest using 4-byte integers for document ids and 4-byte integers for the document term frequency. This is easy, and will let your code work on larger files. However, 4-bytes is more representation than you need for this assignment. I suggest that you store the length of the postings list (*i.e.*, also known as document frequency) with the other information for an indexing term in your dictionary data structure.

Note, the assignment is to build a **non**-positional index as described in Chapter 4 of IIR and in the lecture notes; you should ignore term position *within* the documents.

Description (10 points)

Provide up front, a brief written description of what you did and the format of your lexicon and your inverted file. You should also include:

- (1) Program output that indicates the number of documents processed, size of the vocabulary (*i.e.*, number of unique terms), and total number of terms observed in the collection (this same information was also required for the first program).
- (2) The file sizes for your dictionary and the inverted file (in bytes). Is your index (dictionary + inverted file) smaller than the original text? Which takes up more space, the dictionary or the inverted file?
- (3) Output for the test cases requested in the “Testing” section (below).

¹ Note, the *dictionary* does not have to be written as a binary file, though it would not be unusual to do so.

² If you choose not to make your inverted file binary, there is a 15 point deduction. You should use primitive functions that write integers; you should not use built-in serialization packages (e.g., pickle in Python; writeObject in Java) for the postings lists. For example, if using Java, consider the class RandomAccessFile and the writeInt method in DataOutputStream for the inverted file. In Python, you might find the *struct* and *io* libraries helpful.

Testing (30 points)

Demonstrate the ability to identify which documents a word occurs in and the number of times that the word occurs in each. For full credit do this by reading from your binary inverted file after you have created the index. Doing this in a separate program would make it clear that you are in fact loading the information from disk.

1. Print out the document frequency and postings list for terms: "panda", "python", "Egyptian", "Wyoming". For example, for each term you might output something like: 3 [(123, 1), (456, 2), (789, 1)].
2. Give document frequency for these terms, but do **not** print out their postings lists: "Hopkins", "Stanford", "Brown", and "college" (these postings lists are longer -- here I only want to see the postings list **length**).
3. Print out the docids for documents that have both "Tony" and "Blair" in the text. You can do this by finding the postings list for each term, and then intersecting the two lists. For this test you do not need to use or supply frequency information. Please print the docids in increasing order.

Looking ahead

- For the third programming assignment you will be given queries with the goal of ranking documents using a similarity metric such as the vector cosine method. To succeed on that assignment, it is absolutely crucial that you are able to reload a lexicon from disk and retrieve a postings list from the inverted file for any specified term.
- Assume that the dataset on the third assignment will be larger (e.g., up to 4 GB of text). I have a Java solution for Program #2 that runs in about 10 seconds. If your program takes a much longer time to build an inverted index (say more than 15 minutes) you are probably doing something that is computationally inefficient.

Submission

Reminder: your submission for Program #2 should be a single PDF file that contains the description and testing output requested above, followed by your source code. Please make sure your name is clearly visible on the first page.